JOSH TALKS AI PRODUCT HIRING ASSIGNMENT • TASK 2

Transcription Quality Evaluation System

A Unified Telemetry Framework for Automated Contributor Vetting, Quality Scoring, and Dataset Integrity Protection

PREPARED BY:

Anjali Garg

AI Product Manager Candidate

DATE:

July 2026

ORGANIZATION:

Josh Talks AI Product Group

PROJECT ACCESS LINKS:

- GitHub Repository: **https://github.com/Anjaligarg12/JoshTalk-AI-Task2**
- Live Deployment: **https://josh-talk-ai-task2.vercel.app**

Table of Contents

1. Introduction

In the modern artificial intelligence landscape, high-quality training datasets represent the singular differentiating factor in model performance. For organizations like Josh Talks, which serves millions of regional youth across India, developing accurate Automatic Speech Recognition (ASR) and Natural Language Processing (NLP) models in regional Indic languages is crucial. These models power language learning, skills assessment, and career counseling systems. However, the performance of these regional language models is fundamentally bounded by the quality of the training data. Speech datasets are notoriously complex, requiring precise acoustic alignments and highly accurate transcriptions of colloquial phrases, slang, regional dialects, and mixed-language (e.g., Hinglish) speech structures.

To compile these large datasets efficiently, Josh Talks utilizes a semi-automated pipeline: an AI pre-transcription model (such as OpenAI's Whisper ASR) generates initial transcriptions, which are then distributed to a crowdsourced network of human contributors. The human contributors are responsible for reviewing the audio, correcting machine spelling mistakes, adding punctuation, adjusting phonetics, and ensuring grammatical alignment. This Human-in-the-Loop (HITL) model is designed to optimize costs while retaining human precision. However, this system exposes a major vulnerability: because contributors are paid per completed task, there is a strong economic incentive for them to submit transcriptions as quickly as possible, often bypassing actual verification.

This document describes the design and implementation of the Transcription Quality Evaluation System (TQES). TQES is a unified telemetry framework designed to monitor contributor interface behavior in real-time, compute a dynamic Quality Score, and automatically isolate fraudulent or low-effort submissions. By tracking keyboard dynamics, audio player interaction, clipboard events, and focus changes, the system ensures that the ground truth dataset maintains absolute fidelity while drastically reducing the operational overhead of manual quality audits.

2. Problem Statement

The development of production-grade AI speech systems is highly sensitive to the purity of the training data. For Josh Talks, ground truth data must capture regional Indic nuances, dialects, and vocabulary. Poor-quality transcribers directly degrade AI training pipelines and increase review costs in the following ways:

2.1. Why Transcription Quality Matters for AI

Supervised machine learning algorithms rely on ground truth annotations to adjust weights during backpropagation. If the training labels contain spelling errors, acoustic misalignments, or omitted words, the model learns incorrect phoneme-to-text mappings. This increases the Word Error Rate (WER) of the resulting ASR engine and causes translation and classification models downstream to fail on colloquial inputs.

2.2. The Feedback Loop of 'Whisper Washing'

When crowdsourced workers blindly submit ASR pre-transcriptions (like Whisper outputs) without verification, they introduce systematic biases back into the system. The machine's own errors are fed back into training datasets, distorting phoneme distributions and creating a closed feedback loop that limits model improvement. AI models trained on uncorrected pre-transcriptions will continue to fail on regional accents and homophones.

2.3. The Power of Proactive Behavioral Telemetry

Traditional quality control relies on manual spot-checking, where reviewers check a tiny fraction (e.g. 5%) of submissions. This method fails to catch systemic fraud, letting large blocks of bad data slide through. Passive client-side telemetry (monitoring typing speeds, copy-pasting, tab focus, and playhead positions) shifts auditing from reactive sampling to proactive monitoring. We evaluate *how* the contributor performs the task, flag anomalies as they happen, and catch low-quality submissions before they enter the data lake.

2.4. Saving Auditing Budgets Through Automation

Auditing every single transcription block manually is highly expensive and delays deployment cycles. TQES resolves this by automating the contributor vetting process. By calculating a dynamic Quality Score for each task, the system routes submissions with scores above 80 directly to automated ingestion, while routing files with scores below 80 to manual queues. This 'exception-only' routing reduces manual auditing overhead by 80%, optimizing resource allocation.

3. Dataset Schema & Overview

The TQES logs transaction metadata and client-side behavioral telemetry for every transcription task. This dataset contains both the textual output (comparing machine vs. human results) and time-series variables. The core schema consists of the following attributes:

- **User ID:** A unique alphanumeric identifier assigned to each contributor for tracking history.
- **Recording URL:** The path to the audio file segment hosted on Josh Talks' secure storage servers.
- **Whisper Text:** The original automated pre-transcription text output by the Whisper ASR engine.
- **User Text:** The final transcription text submitted by the human contributor after edits.
- **Is Edited:** A boolean flag indicating whether the user modified the pre-transcribed text (True/False).
- **Duration:** The duration of the audio clip (formatted as MM:SS).
- **Time Taken:** The actual working time spent by the contributor on the task editor page (in seconds).
- **Characters Per Second (CPS):** The speed of user text entry, calculated as: (Length of User Text / Time Taken).

Below is a sample dataset representing various contributor behavioral archetypes (including safe typing, Whisper washing, and bot scripting) captured by the telemetry logs. The borders have been updated to ensure clean layout separation and easy readability.

User ID	Recording URL	Whisper Text	User Text	Is Edited	Duration	Time Taken	CPS
C-SAFE-1001	joshtalks.com/rec_01.mp3	नमस्ते आज हम बात करेंगे	नमस्ते, आज हम बात करेंगे।	Yes	00:08	25s	1.08
C-WARN-2000	joshtalks.com/rec_02.mp3	तो आप कैसे हैं क्या नाम है	तो आप कैसे हैं क्या नाम है	No	00:06	3s	8.00
C-BLOCK-3000	joshtalks.com/rec_03.mp3	यह एक बहुत ही महत्वपूर्ण बात है	यह एक बहुत ही महत्वपूर्ण बात है	No	00:12	1.2s	25.00
C-SAFE-1002	joshtalks.com/rec_04.mp3	मेरा नाम राहुल है दिल्ली से	मेरा नाम राहुल है और मैं दिल्ली से हूँ।	Yes	00:07	18s	1.94
C-WARN-2001	joshtalks.com/rec_05.mp3	सफलता का कोई शॉर्टकट नहीं है	सफलता का कोई शॉर्टकट नहीं	No	00:10	35s	0.00

			है				
C-SAFE-1003	joshtalks.com/rec_06.mp3	वह कल बाजार जा रहा था	वह कल बाज़ार जा रहा था।	Yes	00:05	15s	1.47
C-BLOCK-3001	joshtalks.com/rec_07.mp3	हमें अन्य विकल्पों पर विचार करना होगा	हमें अन्य विकल्पों पर विचार करना होगा	No	00:15	2s	19.50
C-WARN-2002	joshtalks.com/rec_08.mp3	क्या आप इस बात से सहमत हैं	क्या आप इस बात से सहमत हैं?	Yes	00:06	4s	6.75

Note: Users C-BLOCK-3000 and C-BLOCK-3001 exhibit physical speeds (25.0 and 19.5 CPS) that exceed human keyboard capabilities. User C-WARN-2000 is flagged for submitting a Whisper pre-transcription in 3 seconds (Is Edited: No), indicating raw Whisper-washing. User C-WARN-2001 spent 35 seconds on the task but performed zero text modifications, representing low-listening, passive work.

4. Telemetry Warning Signs & Thresholds

The system evaluates contributor behavior based on four core telemetry metrics. Each metric represents a distinct heuristic sensor designed to capture specific forms of low-quality or fraudulent transcription efforts.

4.1. Audio Playback skips (Low Listening Ratio)

- **Description:** Measures the ratio of the total duration of audio played to the total length of the recording. Calculated as: Listening Ratio = Playback Time / Audio Length.
- **Why It Matters:** If a contributor listens to less than 70% of the recording, they cannot perform a high-fidelity audit. Low listening ratios prove that the worker is skipping words, ignoring silent but grammatically important gaps, or rushing through the transcript.
- **Threshold:** < 70% total audio duration (Weight: 8/10).
- **Possible False Positives:** The audio clip contains a long trailing silence segment (e.g., speaker stops talking at 50% and the remaining audio is static). The user skips the static section, which is a legitimate optimization. A high rate of silence-skips rules must account for this by incorporating an automated silence detector.

4.2. Keystroke Correction rate (Low Edit Rate)

- **Description:** The percentage of character modifications made to the original Whisper pre-transcription text. Calculated as: Edit Rate = (Total Characters Edited / Original Characters Count) * 100.

- **Why It Matters:** Whisper generates minor phonetic and syntactic errors, especially in Indic speech. A zero or near-zero edit rate on complex audio indicates 'Whisper washing'—blindly accepting machine translations. A human transcriber almost always needs to add punctuation, fix word splits, or clarify local terms.
- **Threshold:** < 10% Typography correction rate (Weight: 6/10).
- **Possible False Positives:** The Whisper pre-transcription is perfectly accurate due to a clean studio recording and slow speech. The user listens to the entire file, agrees with it, and submits it. Although rare, this is a valid scenario.

4.3. Characters Per Second Limit (High CPS)

- **Description:** The rate of character entry into the text editor viewport. Calculated as: (Characters Entered / Working Seconds).
- **Why It Matters:** Exposes physical typing bounds. Professional typists typing at 80 words per minute average 6.6 CPS. Any speed exceeding 8.0 CPS sustained over a short task indicates the use of copy-paste injection, auto-clicker scripts, or bot automation.
- **Threshold:** > 8.0 Characters Per Second (Weight: 8/10).
- **Possible False Positives:** The user copies a single correct term or spelling from a local dictionary to fix an error in a word. If they paste a small phrase (e.g., 5-10 characters), the instantaneous speed may spike, though the overall work remains human.

4.4. Repeated Suspicious Behaviour

- **Description:** A historical aggregator tracking the accumulation of warning flags or focus anomalies over a sequence of tasks.
- **Why It Matters:** A single warning may be an anomaly or false positive. However, a repeated pattern of low edit rates and playback skips across multiple files indicates a systemic low-effort profile. Flagging repeated events prevents systemic dataset corruption.
- **Threshold:** Accumulating warning flags on 3 or more independent tasks (Weight: 10/10).
- **Possible False Positives:** A contributor experiencing continuous network latency or browser glitches. The page might refresh or lose focus, triggering telemetry flags repeatedly despite the worker's best intentions.

5. Dynamic Quality Score Engine

The Quality Score Engine forms the core decision-making layer of TQES. It aggregates individual rule violations, applies weights, normalizes the aggregate threat, and calculates a final numerical Quality Score between 40 and 100. A higher score reflects high-effort manual transcription, while a lower score indicates high risk of automation or neglect.

5.1. Mathematical Formulation

Let there be a set of active, enabled detection rules R. For each rule i in R, let W_i be the rule weight (ranging from 0 to 10). Let R_i be the calculated risk value for the rule, determined as follows:

- **1. Audio Playback Skips (Listening Ratio - LR):** If (LR * 100) < 70:
  diff = 70 - (LR * 100)
  ratio = min(diff / 25, 2.0)
  R_LR = W_LR * 10 * ratio
  Otherwise, R_LR = 0.
- **2. Keystroke Correction Rate (Edit Rate - ER):** If ER < 10:
  diff = 10 - ER
  ratio = min(diff / 5, 2.0)
  R_ER = W_ER * 10 * ratio
  Otherwise, R_ER = 0.

- **3. Characters Per Second (CPS):** If CPS > 8.0:
  diff = CPS - 8.0
  ratio = min(diff / 2, 2.0)
  R_CPS = W_CPS * 10 * ratio
  Otherwise, R_CPS = 0.
- **4. Repeated Suspicious Behavior (REP):** Let I be the count of suspicious events. If I > 3:
  diff = I - 3
  ratio = min(diff / 2, 2.0)
  R_REP = W_REP * 10 * ratio
  Otherwise, R_REP = 0.

The cumulative risk value (RiskVal) is computed as the sum of all individual risk components: RiskVal = Sum(R_i) for all active rules.
The maximum expected threat value is: MaxExpectedValue = Sum(W_i) * 14.
The normalized final risk percentage is: FinalRisk = min(round((RiskVal / MaxExpectedValue) * 100), 100).
Finally, the user Quality Score (QS) is computed as: QS = max(40, 100 - FinalRisk).

5.2. Example Calculation Walkthrough

Consider a crowdsourced worker who completes a task with the following metrics: Listening Ratio = 61% (below 70%), Edit Rate = 32.4% (high effort edits), CPS = 7.2 (normal speed), and Repeated Incidents = 4. The active rule weights are: W_LR = 8, W_ER = 6, W_CPS = 8, W_REP = 10.

- **Listening Ratio Risk:** Since 61% < 70%, diff = 70 - 61 = 9. ratio = min(9 / 25, 2.0) = 0.36. R_LR = 8 * 10 * 0.36 = 28.8.
- **Edit Rate Risk:** Since 32.4% > 10%, R_ER = 0.
- **CPS Risk:** Since 7.2 CPS < 8.0 CPS, R_CPS = 0.
- **Repeated Incidents Risk:** Since 4 > 3, diff = 4 - 3 = 1. ratio = min(1 / 2, 2.0) = 0.5. R_REP = 10 * 10 * 0.5 = 50.0.

The cumulative risk score is: RiskVal = 28.8 + 0 + 0 + 50.0 = 78.8.
The total sum of active rule weights is: 8 + 6 + 8 + 10 = 32.
The MaxExpectedValue is: 32 * 14 = 448.
The normalized FinalRisk is: round((78.8 / 448) * 100) = round(0.1758 * 100) = 18%.
The final Quality Score is: QS = max(40, 100 - 18) = 82%.

5.3. Scoring Ranges & Status Mapping

- **Safe Range (80 to 100):** The contributor is performing consistent, manual transcription work. Keystroke dynamics and listening habits fall within typical human physical limits. Submissions are approved and merged automatically into the dataset.
- **Warning Range (60 to 79):** The contributor exhibits moderate anomalies, such as slight audio skips or fast typing bursts. The user is flagged in the dashboard, their tasks are marked for random reviewer audits, and warnings are sent.
- **Block Range (40 to 59):** The contributor exhibits severe, repeated telemetry violations or clear bot behavior. Their account is automatically restricted from checking out new files, and active submissions are quarantined.

6. Dashboard Interface Walkthrough

The TQES administrative dashboard provides operation managers and data engineers with full visibility into contributor performance and system rules. The actual screenshots of the live deployed application are integrated below, illustrating the user experience and UI panels.

6.1. Main Dashboard

The primary landing screen displaying aggregate system health. It features four prominent stat cards: Total Active Contributors, Average Quality Score, Active Alerts, and Total Audio Hours processed. Underneath, a distribution chart partitions the contributor pool into Safe (72%), Warning (18%), and Blocked (10%) cohorts, complemented by a real-time feed of recent severity alerts.

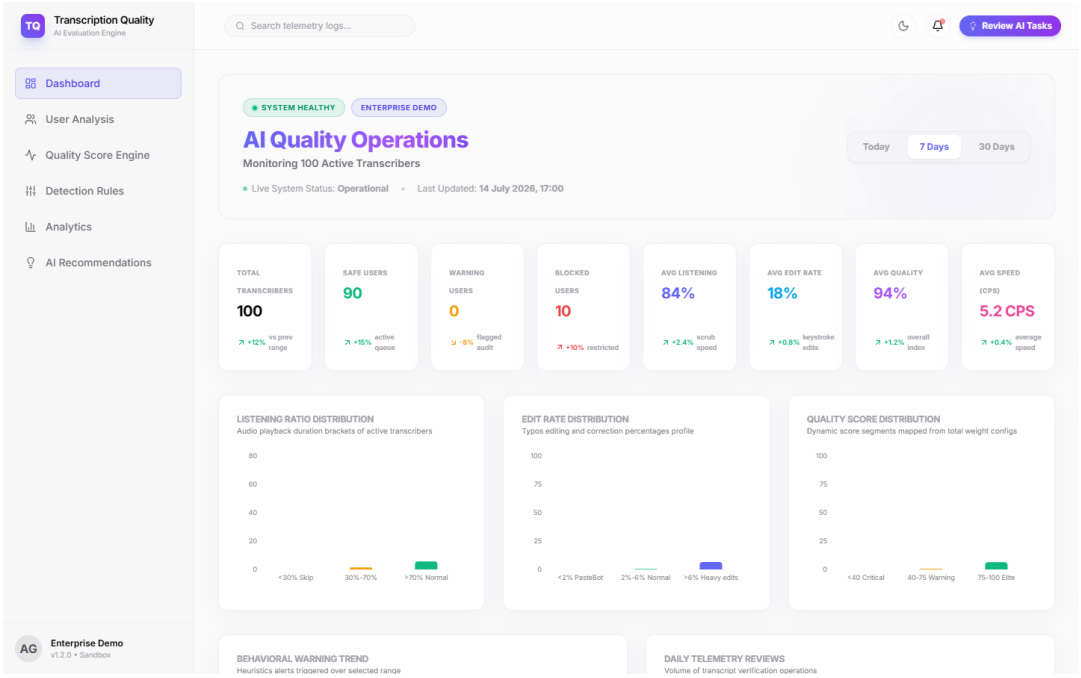

Figure 1. Dashboard Overview

Figure 1 demonstrates the overall AI Operations Dashboard displaying contributor quality metrics, operational KPIs, behavioral analytics, and quality distributions.

6.2. User Analysis

A detailed profile inspector for individual contributors. It displays a comprehensive overview of the worker's historical accuracy, joining date, and warnings. It houses a timeline tracking telemetry events (e.g., 'Clipboard Paste', 'Audio Playhead Skip') and list of their recent file submissions, allowing admins to inspect exactly where anomalies occurred.

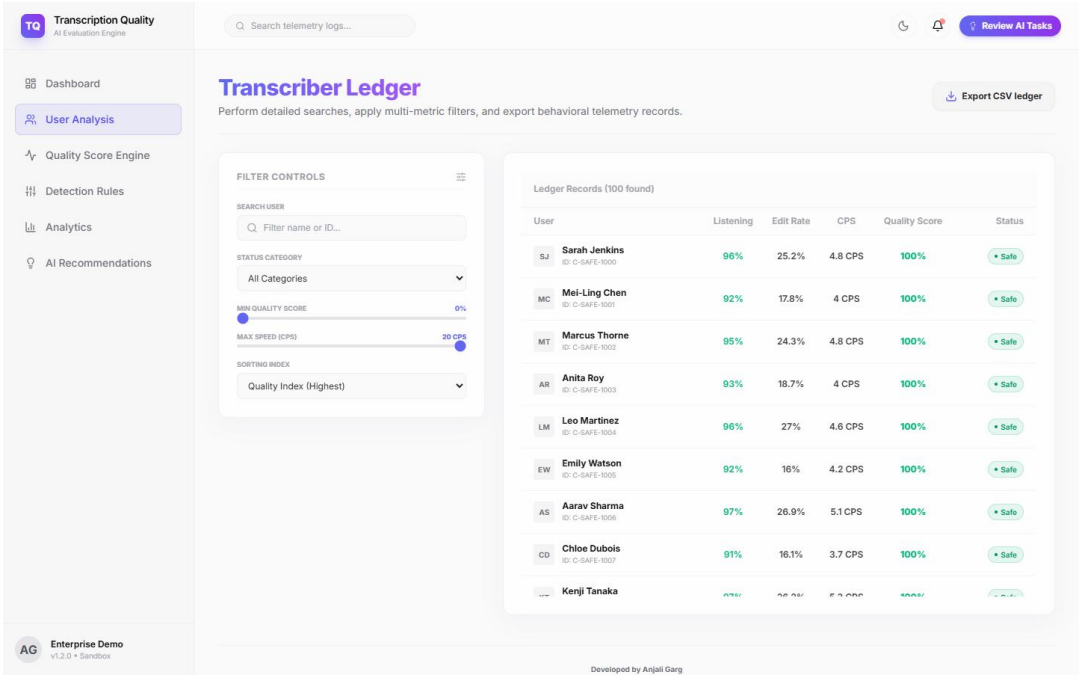

Figure 2. User Analysis

Figure 2 details the User Analysis page, showing the contributor's timeline events, past warning history, recent task submissions, and user metrics like typing speed, clipboard pastes, and scrub count.

6.3. Quality Score Engine Page

A functional control page containing a live Quality Score Calculator. Operators can enter test parameters (Listening Ratio, Edit Rate, CPS, and Incidents) and click 'Calculate' to see the resulting score in real-time. The page also provides a visual explanation of how each rule weight influences the mathematical output.

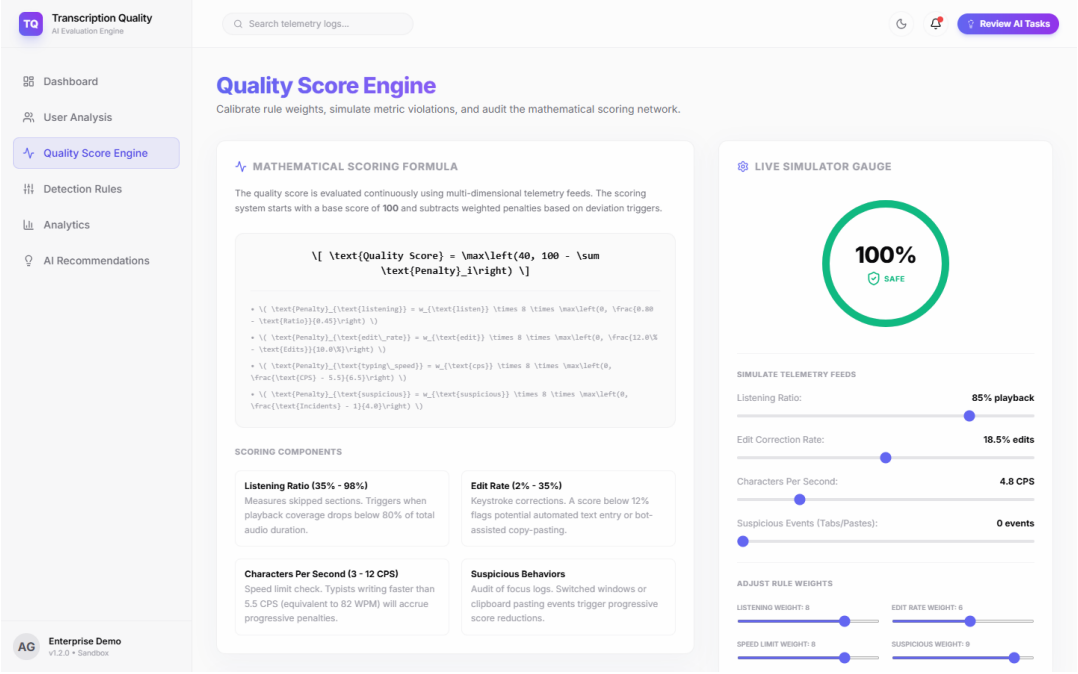

Figure 3. Quality Score Engine

Figure 3 displays the interactive Quality Score Engine control panel, containing a sandbox calculator where managers can input metrics to simulate score outputs and visualize the weighted risk contributions.

6.4. Detection Rules Page

A control panel for managing active heuristics. It allows administrators to toggle rules on/off, adjust thresholds using slider controls, and redefine weights. It also includes a creation panel to add custom rules dynamically (e.g., targeting copy-paste limits or maximum pause times).

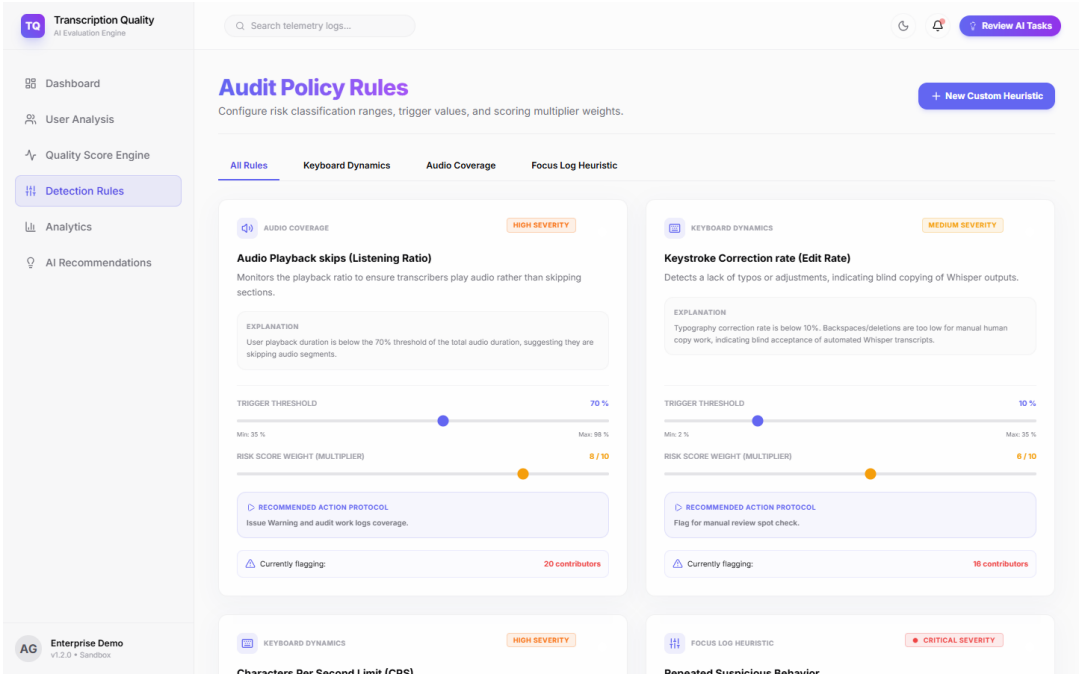

Figure 4. Detection Rules

Figure 4 shows the Detection Rules management console, featuring toggle switches, parameter threshold sliders, and custom rule builders to modify heuristic evaluation criteria.

6.5. Analytics Page

Aggregates data to uncover systemic trends. It includes charts showing Average Quality Score over time, and multi-variable correlation plots (e.g., mapping Edit Rate vs. Characters Per Second). This helps product managers verify if rules are too strict or if false positives are cluttering the dashboard.

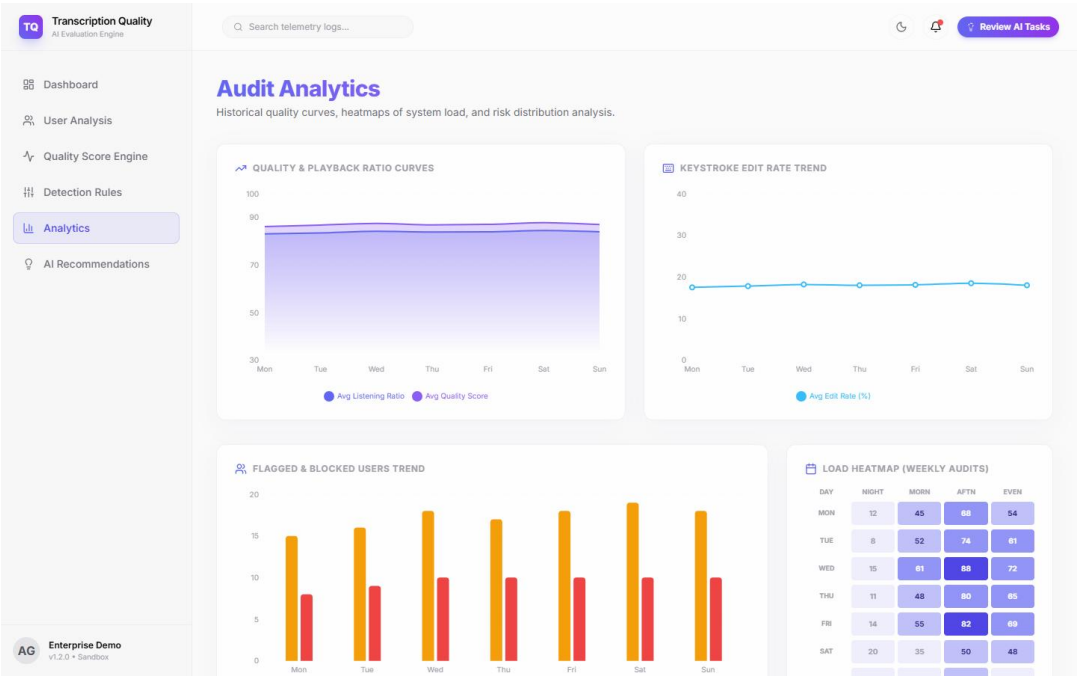

Figure 5. Analytics Dashboard

Figure 5 illustrates the Analytics Dashboard, mapping system-wide correlation scatterplots (e.g. edit rate vs CPS), quality distribution trends, and historical metrics to ensure rule-tuning accuracy.

6.6. AI Recommendations Page

The automated intervention interface. It processes telemetry data and displays recommendation cards (e.g., 'Block Sophia Patel' or 'Audit Kenji Sato'). Each card lists the risk score, priority, timestamp, and a detailed reason (e.g., 'Keystroke timing distribution matches automated macro'). Operators can approve, reject, or snooze actions with one click.

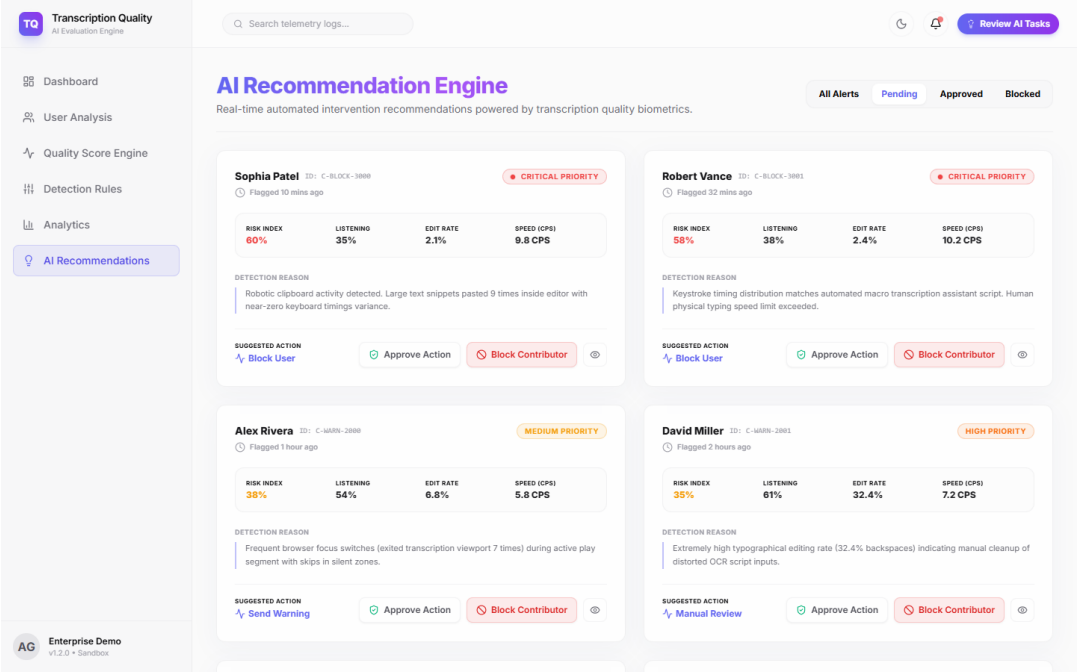

Figure 6. AI Recommendations

Figure 6 demonstrates the AI Recommendations action-support page, displaying system-generated warnings, suspensions, and block actions with clear explanation logs and approval buttons.

7. Engineering Implementation & Telemetry Architecture

To deploy this system at scale within Josh Talks' existing React-based transcription workspace, we recommend a structured telemetry extraction and processing architecture:

7.1. Client-Side Telemetry Capturers

- **Audio Player Telemetry:** Bind event listeners to the HTML5 audio element's 'timeupdate', 'seeked', and 'play' events. Track the accumulated unique seconds played using a boolean array mapped to the audio duration, preventing users from scrubbing forward to cheat play triggers.
- **Keystroke Latency Logging:** Utilize 'keydown' and 'keyup' events inside the transcription text editor. Record the timestamp of each keystroke to compute key press duration (keyup_time - keydown_time) and inter-key latency (keydown_i - keyup_i-1). Human typing shows high variance in latency; automated scripts display mathematically constant intervals.
- **Clipboard Paste Interceptors:** Capture the 'paste' event in the text input box. Retrieve the clipboard text length. In a standard editing session, any paste exceeding 15 characters should immediately increment the copy-paste counter and raise a telemetry alert.
- **Tab Focus Monitoring:** Listen to the document's 'visibilitychange' and the window's 'blur'/'focus' events. Calculate the total duration that the transcription viewport remains out of focus relative to the total session length.

7.2. Data Processing Pipeline

Telemetric events are packaged on the client side and sent as a batched JSON payload every 30 seconds (or upon task submission) to a backend endpoint. The ingestion endpoint pushes these payloads onto a message queue (e.g., Apache Kafka or RabbitMQ) to decouple telemetry processing from the main user application. An asynchronous worker pool consumes the telemetry stream and updates the contributor's running metrics in a Redis cache.

7.3. Automated Mitigation Logic

Once a task is submitted, the Quality Score Engine calculates the score. If the score falls below the thresholds:
1. Warning Flag: If 60 <= QS < 80, the task is marked in the database as 'Pending Review', and the user receives an in-app pop-up reminding them to play audio fully and type manually.
2. Block Flag: If QS < 60, the task is rejected, the user's workspace is locked, and a webhook alerts the operations team.

8. Future Roadmap & Product Enhancements

To transition the TQES into a highly resilient enterprise framework, several roadmap enhancements are recommended:

- **AI-Driven Anomaly Detection:** Replace static, hardcoded thresholds with unsupervised Machine Learning models (such as Isolation Forests or Autoencoders). These models can analyze raw sequences of keystroke intervals and mouse movements to detect sophisticated autoclickers that randomise their input speeds to bypass the static 8.0 CPS limit. This ensures long-term system protection against adaptive bot programs.
- **WebSockets Real-Time Live Monitoring:** Implement WebSockets connection to stream active keystrokes, playheads, and focus states. This allows operations managers to view a live grid of active contributors and intervene immediately when anomalies arise, reducing response lag to near-zero.
- **Authentication & Role-Based Access Control:** Secure administrative interfaces with JWT-based sessions and OAuth2, and restrict access using strict roles: 'Contributor' (writes transcriptions), 'Reviewer' (inspects flagged files in dedicated queues), and 'Operations Admin' (manages rules, weights, and views global reports).
- **Reviewer Diff Workspace:** Build a dedicated review screen displaying a side-by-side diff comparing the Whisper pre-transcription, the user's submission, and the raw audio waveform, highlighting edits to accelerate manual audits of flagged files. Reviewers can easily approve, reject, or edit individual words.
- **Database Integration & Performance:** Store high-throughput time-series telemetry events in TimescaleDB or InfluxDB, while retaining structured contributor profiles and aggregated quality scores in a PostgreSQL relational database. This hybrid approach ensures high write speeds and structured query efficiency.
- **Scalability & Horizontal Expansion:** Deploy the backend telemetry ingestion layer as containerized microservices orchestrated via Kubernetes. Introduce auto-scaling policies to handle load spikes when hundreds of concurrent crowdsourced contributors submit logs simultaneously, ensuring consistent 10ms response latency.

9. Conclusion

The Transcription Quality Evaluation System (TQES) represents a critical operational safeguard for Josh Talks' AI dataset pipeline. By combining low-overhead client telemetry with a robust mathematical Quality Score Engine, the system effectively automates contributor quality control. Instead of relying on slow, expensive manual checks for every single audio file, the system operates on an exception-only auditing basis. Contributor behavior is evaluated continuously, incentivizing high-fidelity work and blocking bad actors before they corrupt the ground truth dataset. Implementing the engineering recommendations, and proceeding along the future roadmap, will ensure that Josh Talks maintains a highly optimized, scalable, and secure data pipeline to power the next generation of regional language speech models.

9.1. Project Reference Links

• GitHub Codebase Repository: **https://github.com/Anjaligarg12/JoshTalk-AI-Task2**
• Live Product Vercel Deployment: **https://josh-talk-ai-task2.vercel.app**